

WinniKnight: MindGuardians

Documentacion de Integracion Funcional — Android

Entrega 4 — Paso 4 | Fundamentos de Aplicaciones Web | Grupo 3

Integrante	Carne	Correo
Pablo Urbina	24001508	24001508@galileo.edu
Angel Camargo	24003664	angel.camargo@galileo.edu

1. Descripcion del Proyecto

WinniKnight: MindGuardians gamifica el bienestar fisico y mental. El usuario derrota monstruos que representan malos habitos (sedentarismo, estres, deshidratacion) realizando acciones reales de salud. La app Android esta construida con Jetpack Compose y conectada a Firebase Firestore y Gemini AI a traves de Firebase Cloud Functions.

2. Arquitectura del Sistema

El sistema sigue una arquitectura de tres capas:

- App Android (Kotlin/Compose) — interfaz de usuario y logica de estado
- Firebase — autenticacion (Auth) y base de datos (Firestore)
- Firebase Cloud Functions (Python) — backend que protege la API key de Gemini

La API key de Gemini nunca esta en el codigo de la app. Vive en Firebase Secrets y solo es accesible desde el servidor de Cloud Functions.

3. Stack Tecnologico

Capa	Tecnologia
UI	Jetpack Compose + Material3

Capa	Tecnologia
Lenguaje	Kotlin 2.0.0
Estado	ViewModel + MutableState
Autenticacion	Firebase Auth
Base de datos	Firebase Firestore
Backend / IA	Firebase Cloud Functions (Python 3.13)
IA	Gemini API (gemini-2.5-pro)
HTTP (Android)	Retrofit 2.11.0 + OkHttp 4.12.0
Build	Android Gradle Plugin 8.5.2

4. Dependencias

4.1 Android (gradle/libs.versions.toml)

Libreria	Version	Uso
firebase-bom	33.1.0	Gestion de versiones Firebase
firebase-auth-ktx	BOM	Autenticacion de usuarios
firebase-firestore-ktx	BOM	Base de datos en tiempo real
retrofit	2.11.0	Cliente HTTP para Cloud Function
converter-gson	2.11.0	Serializacion JSON automatica
okhttp	4.12.0	Control de timeouts HTTP
google-services plugin	4.4.2	Conexion con Firebase

4.2 Cloud Function (functions/requirements.txt)

Libreria	Version	Uso
firebase-functions	~0.5.0	Framework de Cloud Functions
firebase-admin	~6.5.0	Acceso a servicios Firebase desde servidor
requests	~2.31.0	Llamadas HTTP a Gemini API

5. Estructura de Firestore

La siguiente estructura es compartida entre la app Android y la plataforma Web. Ambas plataformas leen y escriben los mismos documentos.

```
users/  
  {uid} /
```

```

displayName: String
email:      String
heroLevel:  Number
heroXp:     Number
heroGold:   Number
heroHp:     Number
totalXp:    Number
battles/
  {battleId}/
    date:      Timestamp
    habitType: String
    result:    String
    goldEarned: Number
    xpEarned:  Number

```

6. Estructura del Proyecto Android

Archivo	Descripcion
data/FirebaseRepository.kt	Firestore: leer/escribir usuario y batallas
data/GeminiRepository.kt	HTTP: llamada a Cloud Function del Oraculo
AuthViewModel.kt	Logica de autenticacion (login/registro)
GameState.kt	ViewModel principal: estado del juego y Firestore
MainActivity.kt	Entry point, navegacion entre pantallas
ui/screens/LoginScreen.kt	Pantalla de inicio de sesion
ui/screens/RegisterScreen.kt	Pantalla de registro con validaciones
ui/screens/Screens.kt	Dashboard, Tienda, Ranking, Oraculo
functions/main.py	Cloud Function: endpoint del Oraculo

7. Flujos Funcionales

7.1 Flujo de Autenticacion

- App inicia: MainActivity verifica si hay sesion activa en Firebase Auth
- Sin sesion: navega a LoginScreen
- Login exitoso: navega a la pantalla principal de combate
- Sin cuenta: navega a RegisterScreen
- Registro: verifica nombre de heroe unico en Firestore, crea usuario en Auth, crea documento en Firestore con stats iniciales
- Login automatico tras registro exitoso

7.2 Flujo de Combate con Firestore

- Usuario presiona boton de ataque
- GameState.attack() actualiza el estado local del monstruo
- Monstruo derrotado: GameState.continueAfterVictory() actualiza gold, xp y level localmente
- FirebaseRepository.saveBattle() escribe el resultado en users/{uid}/battles/
- FirebaseRepository.saveUserData() actualiza users/{uid} con el nuevo estado del heroe

7.3 Flujo del Oraculo (Gemini AI)

- Usuario escribe una hazana de salud y presiona Consultar
- GameState.consultOracle() agrega el mensaje del usuario al chat inmediatamente
- GeminiRepository.consultOracle() hace POST a la Cloud Function
- La Cloud Function recibe el mensaje, construye el prompt RPG y llama a Gemini API
- Gemini responde con narracion epica en espanol
- La respuesta se agrega al chat del Oraculo

8. Configuracion e Instalacion

8.1 Requisitos

- Android Studio Hedgehog o superior
- Android SDK 26+
- Cuenta de Firebase con proyecto MindGuardians configurado
- Firebase CLI instalado: npm install -g firebase-tools
- Python 3.13

8.2 Pasos de instalacion

- Clonar el repositorio
- Abrir la carpeta MindGuardians en Android Studio
- Colocar google-services.json en app/ (descargarlo de Firebase Console)
- Sincronizar Gradle
- Ejecutar en emulador o dispositivo (API 26+)

8.3 Configurar API key de Gemini

La API key nunca va en el codigo. Se configura como Firebase Secret:

```
firebase functions:secrets:set GEMINI_API_KEY
```

8.4 Deploy de Cloud Functions

```
firebase deploy --only functions
```

9. Seguridad

- La API key de Gemini vive en Firebase Secrets, nunca en el código fuente
- google-services.json esta en .gitignore y nunca se sube al repositorio
- Firebase Authentication maneja el ciclo de vida de sesiones
- Firestore en modo Test durante desarrollo (30 días)
- Las reglas de seguridad de Firestore se configuraran en la entrega final

10. Pantallas Implementadas

Pantalla	Descripcion
Login	Autenticacion con email y contraseña
Register	Registro con nombre de heroe unico, email y contraseña
Combate	Pantalla principal: stats del heroe, monstruo animado, botones de ataque
Dashboard	Estadísticas semanales, filtros por habito, grafica de barras
Tienda	Items de equipo con sistema de oro
Ranking	Podio top 3 y lista scrollable con posicion del usuario resaltada
Oraculo	Chat con Gemini AI, respuestas epicas con bonificadores de daño